

Quiz 2

601.471/671 NLP: Self-Supervised Models

Spring 2025

Johns Hopkins University

March 2025

Complete all questions.

Use additional paper if needed.

Time: 90 minutes.

Name of student: _____

Q1. Concepts, Foundations and Big Picture

20 points

For each of the following questions, select the most appropriate answer.

1. (1 points) How is word embedding lookup efficiently implemented in modern neural language models?
- ☐ By storing embeddings as linked lists for quick access
 - ☐ By using a binary search tree to locate embeddings
 - ☐ By matrix multiplication between one-hot encoded vectors and an embedding matrix
 - ☐ By converting embeddings to sequential lists for retrieval

Solution: 3

2. (1 points) Subword tokenization like BPE is especially advantageous for which type of languages?
- ☐ Isolating languages with clear word boundaries
 - ☐ Concatenative languages which form words by stringing together morphemes
 - ☐ Languages with simple morphology
 - ☐ English languages

Solution: 2

3. (1 points) Which of the following is NOT a motivation for using subword tokenization (e.g., BPE) in language modeling? (choose all that apply)
- ☐ To handle out-of-vocabulary words effectively
 - ☐ To limit vocabulary size and reduce memory requirements
 - ☐ To completely remove sparsity in all contexts
 - ☐ To better capture morphological patterns in words

Solution: 3

4. (1 points) Which of the following would be most likely to improve the accuracy of a pre-trained BERT model when fine-tuned on downstream tasks?
- ☐ Extending the pre-training duration with additional unlabeled data relevant to the target domain
 - ☐ Pre-training with a larger lexicon size
 - ☐ Having it only do left-to-right (unidirectional) encoding
 - ☐ Instead of masking tokens at the input and feeding in [MASK], replacing them with random other words and feeding those words

Solution: 1

5. (1 points) You are training an LM for a highly inflected language with many rare word forms. Which tokenization approach would best handle unseen or rare words while keeping the vocabulary size and tokenized sequence length manageable?
- ☐ Use each character as a token (character-level model)
 - ☐ Use each word as a token, with an out-of-vocabulary token for unknown words
 - ☐ Apply a sub-word tokenization method to split words into frequent substrings
 - ☐ Restrict the model to a fixed list of common words and ignore all other words

Solution: 3

6. (1 points) Which of the following statements is INCORRECT?
- ☐ Recurrent neural networks can handle a sequence of arbitrary length
 - ☐ Training recurrent neural networks is hard because of vanishing and exploding gradient problems
 - ☐ Gradient clipping is an effective way of solving the vanishing gradient problem
 - ☐ Recurrent neural networks are commonly used for tasks involving sequential data

Solution: 3 - Gradient clipping is only a solution for solving exploding gradient problems, not vanishing gradient ones.

7. (1 points) Which of the following is a key limitation of training standard RNNs using Backpropagation Through Time (BPTT)?
- ☐ They require explicit feature engineering for word embeddings
 - ☐ They cannot model variable-length sequences
 - ☐ They cannot handle supervised learning tasks
 - ☐ The process of unrolling the network prevents parallel computation

Solution: 4

8. (2 points) Which is/are advantages of Transformers over RNNs for language modeling? (choose all that apply)
- ☐ Transformers can process all positions in a sequence in parallel during training (thanks to self-attention), while RNNs must process tokens sequentially
 - ☐ Transformers capture long-range context more effectively (each token can attend to any other token directly), mitigating the vanishing-gradient and forgetting issues that RNNs have with long sequences
 - ☐ Transformers have fewer parameters than an equivalently powerful RNN, making them less prone to overfitting
 - ☐ Transformers do not require any positional information, since attention alone encodes word order inherently
 - ☐ During training Transformers avoid recursion altogether, whereas RNNs require recursion.

Solution: 1,2,5

9. (1 points) Position encoding in Transformers is primarily used to compensate for which architecture characteristic?
- ☐ Lack of sequence order awareness
 - ☐ Excessive computational cost
 - ☐ Low parameter efficiency
 - ☐ Inability to handle large vocabularies

Solution: 1

10. (1 points) In the Transformer's self-attention mechanism, how is the representation of each word (token) in a layer computed?
- ☐ By using only the embedding of the previous token in the sequence (the token to the left of it)
 - ☐ By a weighted sum of all token embeddings in the sequence, where the weights indicate the relevance of each other token to the current one
 - ☐ By selecting the single most similar token in the sequence and copying its embedding
 - ☐ By taking the uniform average of the embeddings of all tokens in the sequence

Solution: 2

11. (1 points) If we increase the softmax temperature $T > 1$ during text generation, what is the effect on the model's output distribution?
- ☐ The distribution becomes more uniform (flatter), making the next-word choice more random and diverse
 - ☐ The distribution becomes more peaked on the highest-probability token, making the output more deterministic
 - ☐ The probabilities of all tokens remain unchanged
 - ☐ It amplifies the probability of rare tokens more than frequent ones, while keeping the highest token the same
 - ☐ With higher temperature your GPU may start to melt!

Solution: 1

12. (2 points) Your language model's outputs are grammatical but often bland or repetitive when using standard decoding. Which of the following methods can help increase the diversity of generated text compared to greedy decoding while maintaining grammaticality? (choose all that apply)
- ☐ Set the temperature to zero to make the model ultra-confident in a few top choices for each token
 - ☐ Use top- p nucleus sampling with p set near 0 to introduce randomness by limiting to a subset of likely words and sampling among them
 - ☐ Use top- k with $k = 1$ sampling to introduce randomness by limiting to a subset of likely words and sampling among them
 - ☐ Random sampling from the full distribution: at each step, sample the next token according to the model's probabilities without bias towards the top choice
 - ☐ Use top- p nucleus sampling or top- k sampling to introduce randomness by limiting to a subset of likely words and sampling among them
 - ☐ Use a higher softmax temperature ($T > 1$) to make the probability distribution flatter, so lower-probability words have a better chance of being picked

Solution: 4,5,6

13. (1 point) What is the purpose of positional embeddings in transformer models?
- ☐ To adjust the scale of gradients during backpropagation.
 - ☐ To provide consistent positional information for input sequences.
 - ☐ To create sparsity in the attention mechanism for computational efficiency.
 - ☐ To directly enhance the activation functions' output range.

Solution: 2

14. (1 point) How are sine/cosine positional embeddings computed in transformer models?
- ☐ By using a fixed linear transformation matrix applied to each token.

- By generating unique vectors using alternating sin and cos functions based on position (t) and dimension (i).
- By generating unique vectors using alternating sin and cos functions based on position (t).
- By generating unique vectors using alternating sin and cos functions based on dimension (i).

Solution: 2

15. (1 points) Why must even a Transformer LM generate text sequentially at inference time (one token after another), despite the model's parallel processing capability during training?
- Because each next token's prediction requires the previous tokens' output – the model can't know later words until it has generated earlier ones
 - Because GPUs/TPUs do not support parallel computation for Transformer models at inference
 - Because the Transformer switches to a recurrent mode during generation
 - Because the self-attention mechanism only works with one token at a time during inference

Solution: 1

16. (1 points) In Transformer LMs, tying the input and output embeddings can lead to which potential benefit?
- Reduced parameter count and model size
 - Increased prediction diversity
 - Faster convergence
 - More complex training regimes

Solution: 1

17. (1 points) What does Arithmetic Intensity refer to in the context of GPU computing and neural networks?
- The number of floating-point operations per depth level in a neural network.
 - The ratio of mathematical operations to memory accesses in computational tasks.
 - The speed at which data is transferred between different layers of a neural network.
 - The intensity of gradient updates during backpropagation.

Solution: *The ratio of mathematical operations to memory accesses in computational tasks.*

18. (1 points) How does high Arithmetic Intensity benefit the performance of LLMs on GPUs, particularly during self-attention computation?
- It reduces the need for optimization algorithms by increasing data redundancy.
 - It allows for more efficient use of GPU by compute-bound operations.
 - It allows for more efficient use of GPU by memory-bound operations.
 - It allows for a more efficient use of GPU by balancing memory and compute.

Solution: *Option 2; It allows for more efficient use of GPU resources by maximally utilizing the computing power of our GPU (compute-bound operations).*

Q2. Tokenization and Feeding Text to Neural Nets

19 points

1. (3 points) Describe one advantage and one disadvantage of using character-level tokenization compared to subword-level tokenization. If a word is unseen in training data, what happens to it during word-level tokenization? Explain the consequence briefly.

Solution: *Advantage of character-level: It completely mitigates out-of-vocabulary issue because the vocabulary consists only of the alphabet characters.*

Disadvantage of character-level: *It results in much longer token sequences, which increases computational cost and makes modeling long-range dependencies harder.*

With pure word-level tokenization, unseen words become unknown tokens (<UNK>). Consequently, the model treats all unseen words identically, resulting in loss of meaningful distinctions and degraded performance, especially on rare or novel words.

2. (3 points) We tokenized a string and the result came as follows:

```
['Using', 'a', 'Transform', '##er', 'network', 'is', 'simple']
```

What might be the reason that "Transformer" is broken into two subwords? What does "##" signify?

Solution: *The word "Transformer" is broken into two subwords, "Transform" and "##er," likely because of subword tokenization. The "##" indicates that "er" is a continuation of the previous token. This approach efficiently handles large vocabularies and rare words by splitting them into more common subword units.*

3. (3 points) Assume the corpus vocabulary size initially (character-level) was V . After performing N merges in BPE, what will be the new vocabulary size? Express in terms of V, N .

Solution: Initially, vocabulary = V (number of unique characters). After N merges, each merge adds exactly 1 new token. Therefore, new vocabulary size: $V + N$.

4. Consider the following small corpus for training BPE tokenization:

the cat on the mat

(4 points) Show clearly the step-by-step process of BPE tokenization to merge tokens for 2 merges. Start with character-level tokens. Include frequency counts clearly at each step.

Solution:

An example is provided in the slides ...

5. (3 points) What are the typical steps involved in a tokenization pipeline and why is each step needed?

Solution: (1) Normalization: Converts text into a standard format to reduce variability: Strip extra white spaces between words and sentences; Removing punctuations ("Hello, world!" → "Hello world"); Unicode normalization (2) Sentence splitting: Divides text into individual sentences. Uses punctuation marks (., !, ?) and language-specific rules to identify boundaries. (3) The actual tokenization (4) Post processing by truncating to match the maximum length of the model; Pad all sentences in a batch to the same length; Add special token such as EOS. Finally, converts tokens into numerical IDs for model input.

6. (3 points) A model's vocabulary size is 50K. The embedding dimension is 512. What is the size (number of parameters) of the embedding matrix? Explain the implication of this size on the computational resources required. (you may use some math to explain; no need to simplify the calculation).

Solution: Embedding matrix size: $50,000 * 512 = 25,600,000$ parameters. This large embedding matrix significantly increases memory requirements, storage, and computational demands, especially during training, where these parameters must be updated frequently.

Q3. Self-Attention and Vanilla Transformer

23 points

The following code implements "the scaled dot-product attention" in the original Transformer.

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4 import math
5
6 class SimpleSelfAttention(nn.Module):
7     def __init__(self, embed_size, hidden_size):
8         super().__init__()
9         self.hidden_size = hidden_size
10        self.Wq = nn.Linear(embed_size, hidden_size)
11        self.Wk = nn.Linear(embed_size, hidden_size)
12        self.Wv = nn.Linear(embed_size, hidden_size)
13
14    def forward(self, x):
15        q = self.Wq(x)
16        k = self.Wk(x)
17        v = self.Wv(x)
18
19        qk_dot = torch.matmul(q, k.transpose(-2, -1))
20        qk_dot_scaled = qk_dot / math.sqrt(self.hidden_size)
21        qk_dot_softmaxed = F.softmax(qk_dot_scaled, dim=-1)
22
23        output = torch.matmul(qk_dot_softmaxed, v)
24        return output
```

1. (5 points) What is the computational complexity of the forward method in the above? Answer in big O notation with respect to embedding size e , hidden size h , and sequence length n . (Notice that the dimensions—embedding size, hidden size, and the output size—are input parameters set by whoever calling this function.)

Solution: Let embed size, hidden size, sequence length be denoted e, h, n respectively

- (a) The q, k, v vectors take $O(ehn)$ complexity.
- (b) The $q \cdot k$ calculation takes $O(n^2h)$ complexity.
- (c) Dividing by square root of hidden dimension takes $O(n^2)$ complexity
- (d) Softmax takes $O(n^2)$ complexity
- (e) qk dot multiplied with v takes $O(n^2h)$ complexity.

Thus in total, $O(ehn + n^2h)$ compute complexity is needed for the forward step.

2. (3 points) We now want to modify the code above so that tokens in self-attention do not attend to tokens that come after (masking). Describe in plain English how this should be implemented with respect to the code above.

Solution: We should first create a mask in the shape of sequence length by sequence length. The lower triangle should contain 0 and upper $-\infty$. Then we can do elementwise product of values from qk dot scaled with the mask. Other solutions such as setting upper triangle as $-\infty$ should be considered correct.

3. (3 points) Based on the implementation of the given code as-is, is this attention block part of the Transformer encoder, decoder, or something else? How about implementing the masking in the previous question?

Solution: Given code: Encoder, Masked Attention: Decoder

4. (3 points) Explain why scaling the dot product by `math.sqrt(self.hidden_size)` is necessary.

Solution: Scaling the dot product by `math.sqrt(self.hidden_size)` is necessary to prevent large variance in the resulting attention scores. Without scaling, the dot products can become large when the dimensionality of the hidden size is high, leading to extremely small gradients after the softmax function due to saturation. This makes training unstable. By normalizing with the square root of the hidden size, we ensure that the variance of the dot product remains manageable, stabilizing the training process and improving convergence.

5. (3 points) Is the SimpleSelfAttention implementation capable of processing batched inputs x with dimensions `[batch_size, seq_length, embed_size]`? Justify your answer by explaining how batch processing is handled within the implementation.

Solution: Yes, the SimpleSelfAttention implementation can process batched inputs with dimensions `[batch_size, seq_length, embed_size]`. This is because PyTorch operations such as `nn.Linear`, `torch.matmul`, and broadcasting inherently support batch processing. The linear layers (W_q, W_k, W_v) apply transformations independently to each element in the batch, preserving the batch and sequence dimensions. Moreover, `torch.matmul` supports batch-wise matrix multiplication, enabling the computation of attention scores across the entire batch efficiently.

6. (3 points) `k.transpose(-2, -1)` is to transpose the last two dimensions of the tensor k . But why the final two dimensions? Explain why it's implemented this way. (Hint: you can connect it to the previous question)

Solution: Transposing the last two dimensions is crucial for aligning dimensions for matrix multiplication in attention mechanisms. One crucial nuance is that the tensor k typically has dimensions `[batch_size, seq_length, hidden_size]`. So only the final two dimensions need to be transposed and the first dimension needs to remain fixed.

7. (3 points) The command `F.softmax(qk_dot_scaled, dim=-1)` applies softmax along the last dimension. Why is it applied to last dim?

Solution: The operation applies softmax along the last dimension to ensure that all elements within each row (typically corresponding to a sequence position) sum up to 1. This reflects the relative importance or weight each element should have in computing the weighted sum for attention outputs, maintaining consistency across different sequences in the batch.

Q4. Transformer Language Models

38 [+ 3 Extra Credit] points

1. (3 points) Give one example for each of the following types of Transformer-based models:

- Encoder-only: ...
- Encoder-decoder: ...

- Decoder-only: ...

Solution: *Encoder-only: BERT
Encoder-decoder: T5
Decoder-only: GPT
Other models should also be considered correct*

- (3 points) Briefly describe the mathematical definition of ReLU and GeLU activations.

Solution: *The ReLU (Rectified Linear Unit) and GeLU (Gaussian Error Linear Unit) activation functions are defined as follows: (1) ReLU: $\text{ReLU}(x) = \max(0, x)$ This function outputs the input directly if it is positive; otherwise, it outputs zero. It introduces non-linearity into the model and helps mitigate the vanishing gradient problem. (2) GeLU (Gaussian Error Linear Unit): $\text{GeLU}(x) = x\Phi(x)$, where $\Phi(x)$ is the cumulative distribution function of the standard normal distribution. It is often approximated by: $\text{GeLU}(x) \approx 0.5x \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right)\right)$ GeLU smoothly blends positive and negative phase information by incorporating stochasticity through a Gaussian distribution, allowing for more flexible activation compared to ReLU. These functions are integral in neural networks, providing essential non-linearity and facilitating effective learning in LLMs.*

- (4 points) Define what is a KV cache. What are its advantages?

Solution: *A KV (Key-Value) Cache is a method used in transformer models during inference to store key and value representations from prior processing steps. The advantages of using a KV cache include increased efficiency, as it speeds up inference by eliminating the need to recalculate keys and values for tokens that have already been processed.*

- (4 points) Sungwon thinks that caching Q (in addition to KV) would further help increase inference speed. Describe briefly whether this idea is helpful or not (and why).

Solution: *This idea doesn't work. Attention calculation for the $k + 1$ th token only require q_k , not previous query values.*

- (4 points) Explain the difference between serial and parallel formulations of transformer blocks and how this parallelization impacts training speed and model performance.

Solution: *The difference between serial and parallel formulations in transformer blocks lies in the order and method of computation. In the serial formulation, the computation sequence is:*

$$y = x + \text{MLP}(\text{LayerNorm}(x + \text{Attention}(\text{LayerNorm}(x))))$$

Here, attention mechanisms are computed sequentially before passing the results through the MLP layer, with LayerNorm applied at each step. In contrast, the parallel formulation modifies the computation to:

$$y = x + \text{MLP}(\text{LayerNorm}(x)) + \text{Attention}(\text{LayerNorm}(x))$$

In this setup, the MLP and attention computations are independent of each other and can be executed concurrently. LayerNorm is applied only once per input, which is shared by both pathways. The parallel formulation enhances training speed by roughly 15% due to the ability to perform computations simultaneously, thus optimizing resource usage.

- (4 points) Discuss the importance of data cleaning in the preparation of pre-training data for LLMs.

Solution: *By removing irrelevant, erroneous, or duplicate data, the dataset becomes more concise and representative, reducing unnecessary computational load. This leads to faster training times and lower resource consumption.*

- (4 points) The following shows parameters in Llama 3.2 1B Decoder block and their dimensions.

```

1 LlamaDecoderLayer(
2   (self_attn): LlamaAttention(
3     (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
4     (k_proj): Linear(in_features=2048, out_features=512, bias=False)
5     (v_proj): Linear(in_features=2048, out_features=512, bias=False)
6     (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
7   )
8   (mlp): LlamaMLP(
9     (gate_proj): Linear(in_features=2048, out_features=8192, bias=False)
10    (up_proj): Linear(in_features=2048, out_features=8192, bias=False)
11    (down_proj): Linear(in_features=8192, out_features=2048, bias=False)
12    (act_fn): SiLU()

```

```

13 )
14 (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
15 (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
16 )

```

The Q, K, V projection matrices are of different size! What possible modification to the vanilla attention is this attention block using (and why)? (There are multiple possible answers. Providing one possible answer is enough)

Solution: *Either MQA or GQA*

8. (4 points) Diving deeper into docs say that Llama 3.2 1B has 32 query attention heads while having 8 key value heads. Given this added information, what modification of vanilla attention is it using?

Solution: *Since there are more than 1 key value head, they must be using GQA (Group Query Attention)*

9. (3 points) What is the purpose of the (o_proj) layer in the self-attention block, and why does it have different output features compared to the Q, K, V layers?

Solution: *It is responsible for transforming the concatenated attention outputs into the desired output shape. It aligns the output dimension to match the input dimension for residual connections. It often retains the original feature size to maintain consistency within the model architecture.*

10. **Extra Credit** (3 points) Notice that this model is using LlamaRMSNorm normalization. How does this differ from LayerNorm normalization and what are its potential benefits?

Solution: *LlamaRMSNorm normalizes inputs based on the root mean square (RMS) of feature values, unlike traditional layer normalization, which uses mean and variance.*

11. (3 points) Why might the model use eps=1e-05 in LlamaRMSNorm?

Solution: *The eps value of 1e-05 is added to prevent division by zero during normalization. It stabilizes computations by ensuring a minimal variance threshold, which is crucial for maintaining numerical stability during training.*

12. (2 points) Discuss the rationale for the initial expansion to 8192 in up_proj and the subsequent mapping back to 2048 using the down_proj layer.

Solution: *The expansion to 8192 in up_proj introduces more capacity for feature interactions, enabling richer transformations and capturing complex patterns. The subsequent reduction to 2048 in down_proj ensures the output dimension matches the input for residual connections, balancing model expressiveness with architectural consistency.*